

INTRO TO MATLAB

Chapter 1

Copyright © 2017 STEM Course Prep



TOPICS

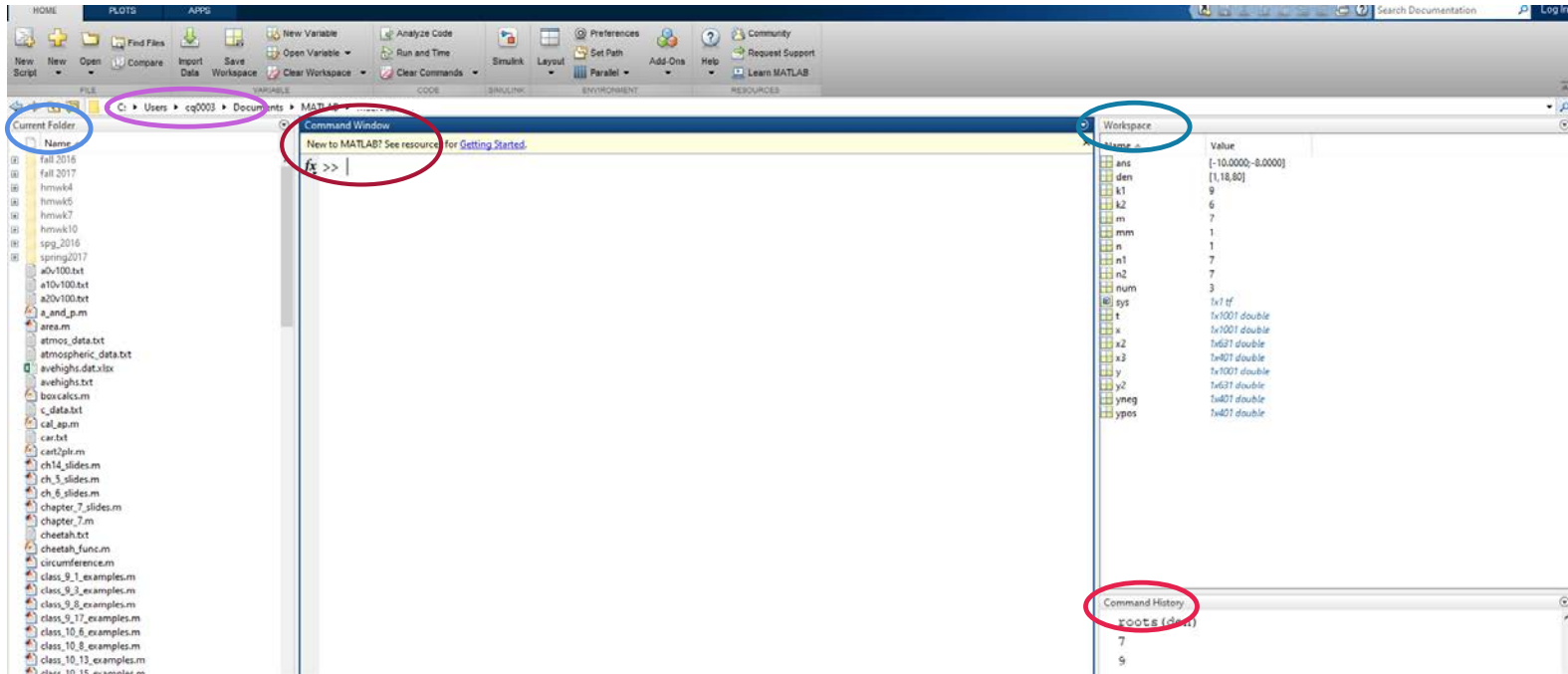
- Desktop Environment
- Variables and Assignments
- Types
- Expressions
- Formatting
- Math Operators

INTRODUCTION TO MATLAB

- Widely-used software package
- Powerful graphing and mathematical functions
- Has programming constructs
- Numerous built-in functions
- Ability to create your own functions/programs

MATLAB DESKTOP ENVIRONMENT

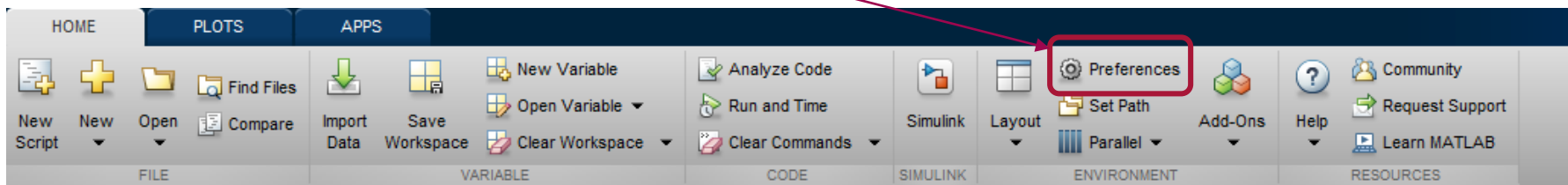
4



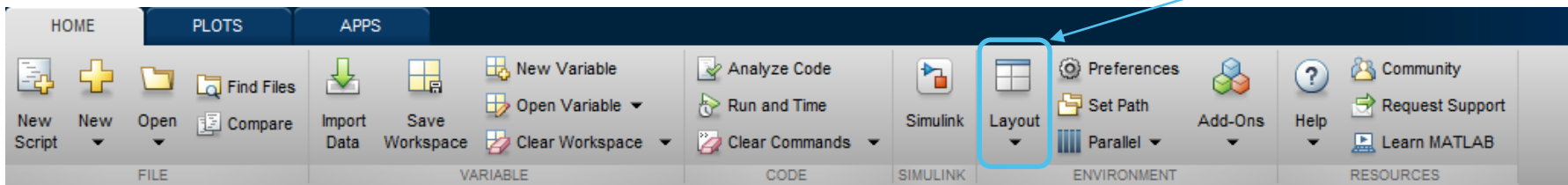
- Current Folder – shows files in current working directory
- Command Window – can use for simple calculations, to run programs, access help menu, etc.
- Workspace – displays information about variables
- Command History – lists recent commands typed in the command window
- Working Directory – Displays location where you are currently working. Files will be saved here.

DESKTOP ENVIRONMENT

- Customize your MATLAB appearance by going to 'Preferences' on toolbar
 - Fonts – can change font type and size
 - Colors – can alter the background and font colors



- Layout of desktop environment
 - Alter the windows and positioning by going to the 'Layout' button on the toolbar



ASSIGNING VARIABLES

- You create a *variable* when you assign a name/letter to a value or expression
- An *assignment statement* can be used to assign a variable
 - General form:
variable = number or expression
 - Keep this order!
 - Name on the left
 - Expression on the right

ASSIGNING VARIABLES

```
Command Window
>> balance=100
balance =
    100
>> balance=100;
>>
>> balance=balance+100
balance =
    200
>> balance=balance*2
balance =
    400
fx >> |
```

Example: You have a balance in the bank of \$100. Do the following:

1. Initialize variable that represents account balance.
 - Notice the variable and value are repeated on the command window after hitting enter
 - To prevent this repetition use a semi-colon (;) at end of line
2. Add 100 to balance
 - balance is now equal to 200
3. Multiply balance by 2
 - balance is now equal to 400
 - $200 \times 2 = 400$

Note: MATLAB goes from top to bottom when doing calculations

ASSIGNING VARIABLES

```
balance =  
    200  
  
>> balance=balance*2  
  
balance =  
    400  
  
>> balance+5  
  
ans =  
    405  
  
>>
```

Workspace	
Name ▲	Value
ans	405
balance	400

- Note that if you don't provide a variable name on the left side, the default variable ans will be used to store the value
- Workspace: notice how the value of balance changes as you modify the variable.
 - MATLAB only stores/uses the last calculated value
- Any variables created will be shown in the workspace

VARIABLE NAMES

- Names must start with a letter of the alphabet
- After that names can contain letters, digits, and the underscore character _
- **MATLAB is case-sensitive**
- Use informative names
 - Names should give an idea what the variable is
 - Generic names like xyz or calc1 are difficult to remember
 - Think about writing a code, forgetting about it, and then coming back to it 4 months from now...will you remember what the variables are? If not, rename them!
- To delete all variables: **clear**
- To delete individual variables: **clear** variable

TYPES/CLASS

10

- **who** and **whos** show available variables

```
>> who
```

```
Your variables are:
```

```
ans      balance
```

```
>> whos
```

Name	Size	Bytes	Class	Attributes
ans	1x1	8	double	
balance	1x1	8	double	

- Variables and expressions have an associated *type* or *class*
 - Floating point/ real numbers: **single**, **double**
 - Integer numbers: no decimal place
 - Signed integers: **int8**, **int16**, **int32**, **int64**
 - Unsigned integers: **uint8**, **uint16**, **uint32**, **uint64**
 - Number indicates the number of bits used for integer
 - Characters and strings (words/text): **char**
 - True/false: **logical**
- Typically use double and int32 types.

TYPE CASTING

- Need to switch from a double to an integer?
- Use **type casting**!
- Use the type names to switch from one to the other
- *Example:* val set equal to a **double** value of 3.7
 - Want to convert 3.7 to a 32-bit integer
 - Type casting converts val to an integer with a value of 4

```
Command Window
>> val=3.7

val =

    3.7000

>> whos
      Name      Size      Bytes  Class  Attributes
      ----      -
      val       1x1           8  double

>> val=int32(val)

val =

    int32

     4

>> whos
      Name      Size      Bytes  Class  Attributes
      ----      -
      val       1x1           4  int32
```

EXPRESSIONS

- Expressions can contain
 - values
 - variables
 - variables must have been initialized prior to being used
 - Remember MATLAB goes from top to bottom
 - operators (see below)
 - built-in functions
 - parentheses
- Operators include:
 - + addition
 - subtraction, negation
 - * multiplication
 - / divided by
 - $21/3 = 7$
 - \ divided into
 - $3 \backslash 21 = 7$
 - ^ exponentiation
 - $5^2 = 25$

FORMATTING ¹³

- **format** changes how values are displayed:
 - short - 4 digits to right of decimal
 - long – 15 digits to right of decimal
 - longg (this one gets rid of scientific notation)
- Other format options:
 - Type: `help format`

```
Command Window
>> format short
>> pi

ans =

    3.1416

>> format long
>> pi

ans =

    3.141592653589793

>> pi*10^10

ans =

    3.141592653589793e+10

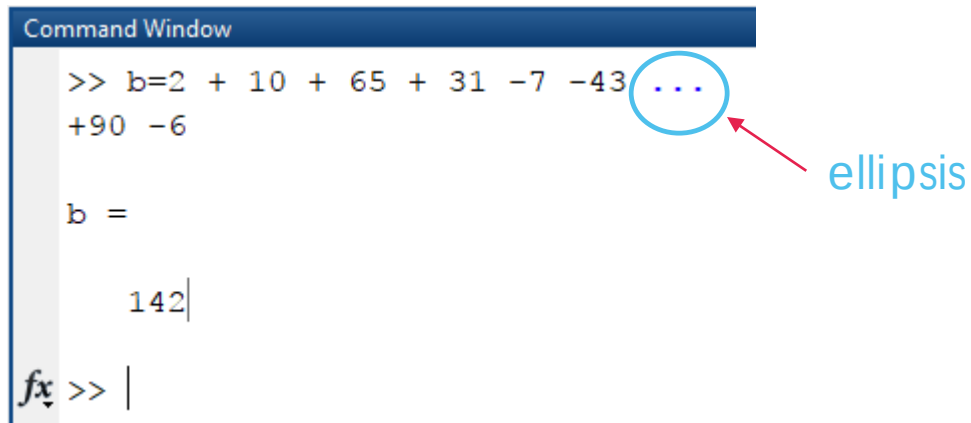
>> format longg
>> pi*10^10

ans =

    31415926535.8979
```

FORMATTING

- Scientific or exponential notation: use e
 - Example: 8e7 equates to 8×10^7
- Have a long expression? Use the ***ellipsis***:
 - Allows all code to be seen without scrolling to the right



The image shows a MATLAB Command Window. The title bar is dark blue with the text "Command Window" in white. The command prompt is ">>". The input is a long arithmetic expression: "b=2 + 10 + 65 + 31 -7 -43 ...". The ellipsis "..." is circled in light blue, and a red arrow points from the word "ellipsis" (written in blue text) to the circle. Below the input, the result is displayed: "+90 -6". Below that, the variable "b =" is shown, followed by the value "142" in a light blue font. At the bottom, the MATLAB logo is followed by ">> |".

```
>> b=2 + 10 + 65 + 31 -7 -43 ...  
+90 -6  
  
b =  
142  
fx >> |
```

ORDER OF OPERATIONS

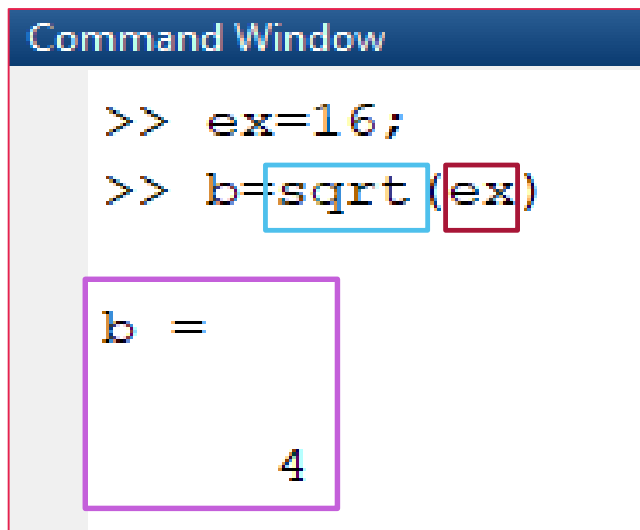
- Just like with algebra, MATLAB has an order of operations
- Order(first to last):
 - () parentheses
 - ^ exponentiation
 - negation
 - *, /, \ multiplication and division
 - +, - addition and subtraction
- Note: expressions in inner parentheses are evaluated first
- Pay close attention to order when creating long expressions

BUILT-IN FUNCTIONS

- One of MATLAB's benefits is the large number of built-in functions
- Built-in functions allow the user to complete a range of tasks quickly and easily
- There are functions that perform simple calculations such as the absolute value
- There are functions that perform complex calculations such as a numerical integration or curve fitting

HOW TO USE FUNCTIONS

- Terminology:
 - *Call* a function by giving the function name followed by *passing* its *argument(s)* in parenthesis
 - If the function calculates a value, it *returns* the value
 - If an assignment statement is used the returned value gets stored in the variable named on the left side



Command Window

```
>> ex=16;  
>> b=sqrt(ex)
```

b =

4

The screenshot shows a MATLAB Command Window. The title bar is dark blue with the text 'Command Window'. The window contains two lines of code: '>> ex=16;' and '>> b=sqrt(ex)'. Below the code, the output 'b =' is displayed, followed by the value '4' on a new line. Three colored boxes are overlaid on the image: a light blue box around 'sqrt' in the code, a red box around 'ex' in the code, and a purple box around the entire output area containing 'b =' and '4'.

Example: sqrt calculates the square root of the argument and returns the value

 function name

 argument

 returns result

ELEMENTARY FUNCTIONS

- Trig functions:
 - **sin, cos, tan, sec, csc, cot**
 - Arguments must be in radians
 - Arc functions: **asin, acos, atan, asec, acsc, acot**
 - Returns angle in radians
 - Want to use degrees instead? Just add a 'd' to the end:
 - **sind, cosd, asind**, etc.
 - Syntax: **cos(x), acosd(x)**
- Exponential functions:
 - exp – exponential (e^x)
 - **exp(-2)** equates to e^{-2}
 - log – natural log ($\ln x$)
 - **log(1)** equates to $\ln 1$
 - sqrt – square root ($\sqrt{x}$)
 - **sqrt(4)** equates to $\sqrt{4}$

ELEMENTARY FUNCTIONS

- Complex functions:
 - **abs** - absolute value ($|x|$)
 - **conj** - complex conjugate
 - **imag** - complex imaginary part
 - **real** - complex real part
- Rounding functions:
 - **round** - round towards nearest integer
 - **ceil** - rounds up towards $+\infty$
 - **floor** - rounds down towards $-\infty$
 - **fix** - rounds towards zero
 - **sign** - gives + or -

GETTING HELP

```
>> help
HELP topics:
```

[Documents\MATLAB](#)
[matlab\datafun](#)
[matlab/datatypes](#)
[matlab\elfun](#)
[matlab\elmat](#)
[matlab\funfun](#)
[matlab\general](#)
[matlab\iofun](#)
[matlab\lang](#)
[matlab\matfun](#)
[matlab\ops](#)
[matlab\polyfun](#)
[matlab\randfun](#)
[matlab\sparfun](#)
[matlab\specfun](#)
[matlab\strfun](#)
[matlab\timefun](#)
[matlab\validators](#)
[matlab\matlabxl\matlabxl](#)
[matlab\demos](#)

- (No table of contents file)
- Data analysis and Fourier transforms.
- Data types and structures.
- Elementary math functions.
- Elementary matrices and matrix manipulation.
- Function functions and ODE solvers.
- General purpose commands.
- File input and output.
- Programming language constructs.
- Matrix functions - numerical linear algebra.
- Operators and special characters.
- Interpolation and polynomials.
- Random matrices and random streams.
- Sparse matrices.
- Specialized math functions.
- Character arrays and strings.
- Time and dates.
- (No table of contents file)
- (No table of contents file)
- Examples.

function group name

To see a list of help topics, use the command `help`

```
>> help
```

To find the functions in a help topic, type `help` with function group name:

```
>> help demos
```

- To find out about a particular function like absolute value:

```
>> help abs
```

CONSTANTS

- Constant – known value that does not change
- MATLAB has built-in constants
 - pi - π
 - i or j - $\sqrt{-1}$
 - Inf, -Inf - $\pm\infty$
 - NaN – not a number
 - You don't want to see this!
 - Means your tried to divide by zero.

RELATIONAL EXPRESSIONS

- Relational operators:
 - > greater than
 - < less than
 - >= greater than or equals
 - <= less than or equals
 - == equivalent
 - ~= not equivalent
- Return a **logical type**:
 - 1 for true
 - 0 for false
- To make more complex comparisons use logical operators:
 - | or
 - & and
 - ~ not